



Deploy

Prof. Lázaro

O que é Deploy?

Deploy é o processo de **colocar uma aplicação em produção** ou em um ambiente acessível. Pode ser local, em servidores, ou em nuvem.



Manual vs Automático

Manual: copiar arquivos via FTP, rodar comandos no servidor.

Automático (CI/CD): um pipeline executa testes, builds e envia ao servidor após cada push.

O que é CI/CD?

CI – Continuous Integration: sempre que alguém envia código, ele é testado automaticamente.

CD – Continuous Delivery: o código testado é preparado para deploy.

CD – Continuous Deployment: o código é **automaticamente** “*deployado*” após os testes passarem.

Por que Docker e GitHub Actions?

Docker isola sua aplicação em containers, garantindo que ela rode igual em qualquer lugar.

GitHub Actions automatiza processos, como testes e deploy, diretamente no repositório GitHub.

Dockerfile

Dockerfile.prod > ...

```
1 FROM node:22.15-alpine AS builder
2
3 WORKDIR /app
4
5 COPY frontend/package*.json ./
6 # RUN npm ci
7 RUN npm i
8
9 COPY frontend/ .
10 RUN npm run build
11
12 FROM nginx:alpine AS runner
13
14 COPY --from=builder /app/dist /usr/share/nginx/html
15
16 # Copia configuração customizada do Nginx (opcional)
17 # COPY nginx.conf /etc/nginx/nginx.conf
18
19 EXPOSE 80
20
21 CMD ["nginx", "-g", "daemon off;"]
```

Dockerfile.dev > ...

```
1 FROM node:22.15.0
2
3 WORKDIR /app
4
5 COPY frontend/package*.json ./
6
7 RUN npm install
8
9 COPY frontend/ .
10
11 EXPOSE 5173
12
13 ENV NODE_ENV=development
14
15 CMD [ "npm", "run", "dev", "--", "--host" ]
```

FROM: imagem base

WORKDIR: diretório padrão dentro do container

COPY: copia arquivos

RUN: roda comandos

EXPOSE: expõe a porta

ENV: variáveis de ambiente

CMD: comando final para rodar a aplicação

```
🐳 docker-compose.yml > ...
  docker-compose.yml - The Compose specificati
1  version: "3.8"
2
  > Run All Services
3  services:
  > Run Service
4    frontend:
5      build:
6        context: .
7        dockerfile: Dockerfile.dev
8      ports:
9        - "5173:5173"
10     volumes:
11       - ./frontend:/app
12       - /app/node_modules
13     environment:
14       - NODE_ENV=development
15     stdin_open: true
16     tty: true
17
18   # prod:
19   #   build:
20   #     context: .
21   #     dockerfile: Dockerfile.p
22   #   ports:
23   #     - "8080:80"
```

Docker Compose

- Define vários containers (ex: front + API + banco) no mesmo arquivo.
- **Variáveis de ambiente**
 - Usadas com ENV no Dockerfile ou .env no Compose.

.github > workflows > deploy.yml > {} on > {} pus

GitHub Workflow - YAML GitHub Workflow (github-workfl

```
1  name: CI/CD
2
3  on:
4    push:
5      branches:
6        - main
7
8  jobs:
9    build-and-test:
10     runs-on: ubuntu-latest
11     defaults:
12       run:
13         working-directory: ./frontend
14
15     steps:
16       - name: Checkout do código
17         uses: actions/checkout@v3
18
19       - name: Setup Node.js
20         uses: actions/setup-node@v4
21         with:
22           node-version: 22.15
23
24       - name: Instalar dependências
25         # run: npm ci
26         run: npm i
27
28       - name: Rodar os testes
29         run: npm run test
```

Fundamentos de GitHub Actions

- Workflows são arquivos .yml em .github/workflows.
- **Trigger:** evento que inicia (ex: push, pull_request)
- **Jobs:** agrupam etapas (steps)
- **Steps:** comandos que rodam
- **uses:** reusam ações da comunidade
- **run:** executa comandos shell
- **secrets:** variáveis protegidas

deploy:

`runs-on: ubuntu-latest``needs: build-and-test``if: success()`

steps:

- name: Checkout do código
uses: actions/checkout@v3

- name: Setup Docker
uses: docker/setup-buildx-action@v3

- name: Login no Docker Hub (opcional, se quiser usar r
uses: docker/login-action@v3
with:
username: \${ secrets.DOCKER_USERNAME }}
password: \${ secrets.DOCKERHUB_TOKEN }}

- name: Testar login no Docker
run: docker info

- name: Build imagem de produção
run: docker build -f Dockerfile.prod -t minha-app:lat

Se quiser mandar para Docker Hub

- name: Push imagem para Docker Hub

run: docker tag minha-app:latest meu-usuario/minha-

Deploy para a Vercel via Vercel CLI

- name: Deploy to Vercel
uses: amondnet/vercel-action@v20
with:

- vercel-token: \${ secrets.VERCEL_TOKEN }}

- vercel-args: "--prod"

- vercel-org-id: \${ secrets.VERCEL_ORG_ID }}

- vercel-project-id: \${ secrets.VERCEL_PROJECT_ID }}

Pipeline de build do container e o deploy

- Se os testes do build and test passar?
- Faz o checkout do Código
- Configura e cria a imagem docker do projeto
- Faz o envio do código para a Vercel

Estratégias de Deploy: Blue-Green Deploy

Conceito:

- Dois ambientes idênticos: **Blue** (produção atual) e **Green** (nova versão).
- A nova versão é deployada no Green.
- Se tudo estiver certo, o tráfego é redirecionado do Blue para o Green.

Vantagens:

- Rollback instantâneo: se algo falhar, é só voltar o tráfego para o Blue.
- Sem downtime perceptível.

Considerações:

- Requer duplicação de infraestrutura.
- Pode ser complexo em bancos de dados (exige migrações compatíveis em ambos os ambientes).

Blue-Green

```
1  # .github/workflows/deploy-blue-green.yml
2  name: Deploy Blue-Green
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v3
13       - uses: actions/setup-node@v3
14         with:
15           node-version: 18
16       - run: npm ci && npm run build
17       - run: docker build -t myapp:${{ github.sha }} .
18
19    deploy-green:
20     needs: build
21     runs-on: ubuntu-latest
22     steps:
23       - name: Deploy to Green environment
24         run: |
25           # Exemplo: atualizar serviço green com nova imagem
26           kubectl set image deployment/myapp-green myapp=myapp:${{ github.sha }}
27
28    switch-traffic:
29     needs: deploy-green
30     runs-on: ubuntu-latest
31     steps:
32       - name: Redirect traffic to Green
33         run: |
34           # Exemplo: alterar DNS ou apontamento de load balancer
35           kubectl patch service myapp -p '{"spec":{"selector":{"env":"green"}}}'
36
```

Canary Deploy

Conceito:

- O deploy é feito **gradualmente**, começando com uma pequena porcentagem de usuários (ex: 5%).
- Monitora-se erros, logs, comportamento do sistema.
- Se não houver problemas, a versão é expandida até 100%.

Vantagens:

- Detecta falhas sem afetar todos os usuários.
- Alta tolerância a bugs.

Considerações:

- Requer balanceamento de carga inteligente (ex: via ingress, proxy, CDN).
- Lógica de roteamento pode ser mais complexa.

Canary

```
1  # .github/workflows/deploy-canary.yml
2  name: Deploy Canary
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    build:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v3
13       - run: npm ci && npm run build
14       - run: docker build -t myapp:canary-${{ github.sha }} .
15
16    deploy-canary:
17     needs: build
18     runs-on: ubuntu-latest
19     steps:
20       - name: Deploy to 10% of users
21         run: |
22           # Exemplo: Kubernetes com selector de 10% de tráfego
23           kubectl apply -f k8s/deployment-canary.yaml
24
```

Rolling Deploy

Conceito:

Atualiza os servidores ou containers **um por um** (ou em lotes), sem parar todo o serviço.

Vantagens:

Evita downtime.
Fácil de automatizar com orquestradores como Kubernetes ou Docker Swarm.

Considerações:

Pode haver inconsistência temporária se múltiplas versões coexistirem.
Precisa que as versões sejam compatíveis entre si.

Rolling

```
1  # .github/workflows/deploy-rolling.yml
2  name: Deploy Rolling
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    rolling-deploy:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v3
13       - run: npm ci && npm run build
14       - run: docker build -t myapp:latest .
15       - name: Push image
16         run: docker push myapp:latest
17       - name: Rolling update
18         run: |
19           kubectl rollout restart deployment/myapp
20
```

Preview Deploy (ou Review Apps)

Conceito:

- Cada pull request gera um ambiente isolado (URL única) para testes manuais ou automáticos.
- Comum em Vercel, Netlify, GitHub Actions + Docker.

Vantagens:

- Permite validar antes do merge.
- Testes com clientes/QA em um ambiente real.

Considerações:

- Gera custo computacional extra.
- Pode exigir controle de acesso para PRs privados.

Preview ou Review

```
1  # .github/workflows/preview-deploy.yml
2  name: Deploy Preview
3
4  on:
5    pull_request:
6      branches: [main]
7
8  jobs:
9    deploy-preview:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v3
13       - run: npm ci && npm run build
14       - run: docker build -t myapp:pr-${{ github.event.pull_request.number }} .
15       - name: Deploy to preview environment
16         run: |
17           # Exemplo com Vercel, Render, Railway ou Docker
18           docker run -d -p 3000:3000 --name=myapp-pr${{ github.event.pull_request.number }} myapp:pr-${{ github.event.pull_request.number }}
```

Feature Toggle (Flags de recurso)

Conceito:

- O código de uma nova funcionalidade é deployado **desligado por padrão**, ativado por configuração.

Vantagens:

- Decouple entre deploy e release.
- Rollback sem novo deploy: só desligar o flag.

Considerações:

- Lógica condicional aumenta a complexidade do código.
- Flags precisam ser gerenciadas (ex: com LaunchDarkly ou Unleash).

Feature Flag

```
1  # .github/workflows/deploy-with-feature-flags.yml
2  name: Deploy with Feature Flags
3
4  on:
5    push:
6      branches: [main]
7
8  jobs:
9    deploy:
10     runs-on: ubuntu-latest
11     steps:
12       - uses: actions/checkout@v3
13       - run: npm ci && npm run build
14       - run: docker build -t myapp:latest .
15       - run: docker push myapp:latest
16       - name: Deploy to production
17         run: |
18           kubectl set image deployment/myapp myapp=myapp:latest
19
```

A feature flag está no código:

```
1  if (flags.newFeatureEnabled) {
2    render(<NewFeature />);
3  } else {
4    render(<OldFeature />);
5  }
```

A flag pode vir do LaunchDarkly, Firebase Remote Config ou config JSON.

Quadro comparativo: Estratégias de Deploy

Estratégia	Características Principais	Vantagens	Desvantagens	Quando Usar
Blue-Green	Dois ambientes idênticos (Blue e Green); tráfego é trocado entre eles	• Rollback imediato • Sem downtime perceptível	• Custo maior (infra duplicada) • Migração de dados precisa ser cuidadosa	• Ambientes críticos • Alto controle de estabilidade
Canary	Deploy gradual para uma pequena porcentagem de usuários	• Reduz riscos • Fácil detectar erros em produção inicial	• Complexidade no roteamento • Requer métricas e monitoramento	• Aplicações com muitos usuários • Lançamentos sensíveis
Rolling	Substitui as versões em produção de forma progressiva (um por vez ou em lotes)	• Evita downtime • Boa escalabilidade	• Possível inconsistência entre versões • Rollback pode ser lento	• Deploys frequentes • Infraestrutura com orquestrador
Preview/Review Apps	Cria ambiente temporário por PR (Pull Request) com URL acessível	• Validação antecipada • Teste visual ou QA externo	• Custo de infraestrutura extra • Pode gerar lixo se não limpo	• Equipes de produto ou QA • Validação com cliente
Feature Toggle	Código vai para produção desativado e é ativado por configuração (flag de recurso)	• Separa deploy de release • Rollback sem redeploy	• Aumenta a complexidade do código • Flags precisam ser bem gerenciadas	• Teste A/B • Releases sob controle de marketing

Boas Práticas para Deploy com Docker

Use multi-stage builds

- Separa a fase de build e de execução.
- Gera imagens menores e mais seguras.

Evite copiar arquivos desnecessários

- Use .dockerignore para ignorar node_modules, tests, .env, etc.

Não rode como root

- Use USER node no final do Dockerfile para segurança.

Cache inteligente

- Copie package.json antes dos arquivos para aproveitar o cache do npm ci.

Use tags imutáveis

- Não use latest em produção. Prefira versões fixas (app:1.2.3).

Boas práticas com GitHub Actions

Separe testes de deploy

- Não faça deploy se os testes falharem.
- Use jobs separados: test, build, deploy.

Use secrets

- Nunca exponha tokens, senhas, chaves diretamente no código.
- Configure no GitHub em Settings > Secrets and variables > Actions.

Aproveite o caching

- Cache do node_modules, vite, pnpm ou yarn acelera builds.
- Uses: actions/cache@v3

Use ambientes (environments)

- GitHub permite definir environments com aprovações manuais, secrets específicos, etc.

Naming consistente

- Nomeie seus workflows claramente (ci.yml, deploy-prod.yml).
- Use name: no topo para clareza nos logs.

Verificações automáticas

- Configure branch protection para exigir que os workflows de teste passem antes de merge.

GitHub Actions Marketplace

- <https://github.com/marketplace?type=actions>
- É o repositório oficial de ações disponíveis para uso em workflows. Nele, você pode:
 - Explorar ações por categorias como CI/CD, testes, segurança, deploy, entre outras.
 - Visualizar detalhes de cada ação, incluindo exemplos de uso, número de estrelas e verificações de segurança.
 - Adicionar ações diretamente aos seus workflows com facilidade.
- Local ideal para descobrir e integrar ações.

Lista de Ações Úteis por Guillaume Falourd

- <https://github.com/GuillaumeFalourd/useful-actions>
- Este repositório no GitHub oferece uma seleção de ações genéricas úteis para diversos casos de uso. Ele inclui:
 - Ações globais para tarefas comuns.
 - Ações específicas para Docker.
 - Ferramentas adicionais para otimizar workflows.
 - Exemplos práticos de como implementar essas ações em seus projetos.
- É uma fonte para encontrar ações testadas e aprovadas por desenvolvedores.